UNIVERSIDAD TECNOLOGICA DE EL SALVADOR

FACULTAD DE CIENCIAS SOCIALES

ESCUELA DE ANTROPOLOGÍA

ASIGNATURA: PROGRAMACIÓN III

SECCIÓN: 03

DOCENTE: ELISEO ERNESTO HERNÁNDEZ DIAZ

ALUMNOS:

Erick Vlasdimir Gaitán Ramírez 25-3677-2020
Justin Brian Sánchez Cardona 25-4195-2024
Wilson Stephen Martínez Alfaro 25-2887-2024
José Ernesto Crespin Sayes 25-0599-2024
David Reynaldo Trejo Jiménez 25-1265-2023
Diego Anthony Ramos Alemán 25-3224-2021
William robert hernandez ascencio 25-2235-2016

Repositorio: https://github.com/erick2599/Investigacion-ControldeGastos

MARCO TEÓRICO Y DISEÑO METODOLÓGICO

Análisis Comparativo Del Rendimiento De Sqlite Y Firebase En Operaciones De Inserción Y Consulta Para Un Gestor De Gastos Personales

Resumen

El presente trabajo propone evaluar comparativamente el rendimiento de SQLite y Firebase como mecanismos de almacenamiento para un gestor de gastos personales en Android. La investigación se delimita a las operaciones de inserción y consulta de registros y considera tres volúmenes de datos: 100, 1.000 y 10.000 registros. La variable principal será el tiempo de respuesta, medido en milisegundos desde el inicio de una operación hasta la confirmación de su finalización. Se realizarán cinco ejecuciones de calentamiento y treinta mediciones por combinación de tecnología, operación y volumen, manteniendo constantes el dispositivo, la versión de la aplicación, la estructura lógica del registro y las condiciones de prueba. En Firebase se registrará también el estado de conectividad, porque el servicio remoto depende de la red; en SQLite se medirá la operación local. El marco se apoya en estudios comparativos previos, documentación técnica de Android, Firebase y SQLite, y el modelo de calidad ISO/IEC 25010:2023. Se reconoce como limitación que el tiempo local y el tiempo de una operación en la nube incluyen rutas de ejecución diferentes; por ello, los resultados se interpretarán como desempeño observado en el escenario definido y no como superioridad universal de una tecnología.

1. Introducción

El registro de gastos personales requiere almacenar transacciones, consultarlas por fecha o categoría y generar resúmenes que ayuden al usuario a comprender sus hábitos financieros. Un gestor de gastos puede utilizar almacenamiento local para responder sin depender de una conexión, almacenamiento en la nube para sincronizar información entre dispositivos o una combinación de ambos. La decisión tecnológica afecta el tiempo percibido por el usuario, la disponibilidad de los datos y la complejidad de implementación.

El repositorio del equipo define como objetivo comparar Firebase y SQLite en operaciones de consulta e inserción con diferentes volúmenes de datos. Para mantener la continuidad con el proyecto existente, esta entrega adopta ese alcance técnico y lo contextualiza en el gestor de gastos personales. La guía de Entrega 2 exige que el marco teórico, la revisión de antecedentes y el método describan procedimientos replicables, con métricas operacionales, control de variables, análisis estadístico y evidencia técnica en GitHub.

El estudio no pretende demostrar qué base de datos es mejor en todos los escenarios. SQLite es un motor relacional embebido que opera localmente; Firebase es una plataforma de base de datos alojada en la nube, cuya latencia puede depender de la red, la ubicación

del servicio, el modelo de datos y la estrategia de sincronización. La comparación será válida únicamente bajo las condiciones documentadas.

1.1 Planteamiento del problema

En un gestor de gastos personales, una demora al guardar una transacción o consultar el historial puede afectar la fluidez de uso. SQLite y Firebase ofrecen arquitecturas diferentes: la primera ejecuta operaciones en el dispositivo, mientras que Firebase requiere considerar el acceso a un servicio remoto, aunque sus SDK pueden incorporar mecanismos de caché o sincronización. Por tanto, el rendimiento observado puede variar según la operación, el volumen de registros y la conectividad.

El problema de investigación consiste en determinar cómo varía el tiempo de respuesta de inserción y consulta al utilizar SQLite frente a Firebase en un prototipo Android de gestor de gastos, bajo volúmenes de datos y condiciones de prueba controlados.

1.2 Pregunta de investigación

¿Qué diferencias se observan en el tiempo de respuesta de las operaciones de inserción y consulta entre SQLite y Firebase para un gestor de gastos personales Android, al trabajar con 100, 1.000 y 10.000 registros bajo condiciones controladas?

1.3 Objetivos

Objetivo general: Comparar el rendimiento de Firebase y SQLite en operaciones de inserción y consulta para un gestor de gastos personales, utilizando diferentes volúmenes de datos bajo condiciones controladas.

Objetivos específicos: (1) Definir una estructura lógica común de registros de gastos para los escenarios de prueba. (2) Implementar o documentar un banco de pruebas que ejecute inserciones y consultas equivalentes en ambos mecanismos. (3) Medir el tiempo de respuesta con múltiples repeticiones para cada combinación. (4) Analizar la variación de los tiempos según tecnología, operación y volumen de datos. (5) Formular recomendaciones de selección basadas exclusivamente en los resultados observados y las limitaciones del experimento.

1.4 Delimitación

Teórica: ingeniería de software, evaluación de calidad, eficiencia de desempeño, almacenamiento local y almacenamiento en la nube; se utilizará ISO/IEC 25010:2023 como referencia de calidad del producto.

Temporal: ciclo académico 02-2026, durante las cuatro entregas de Investigación Formativa.

Contextual: prototipo académico Android de gestor personal de gastos; las pruebas se realizarán en un entorno controlado y no con información financiera real de usuarios.

Técnica: comparación de SQLite local y Firebase en operaciones de inserción y consulta. Se consideran volúmenes de 100, 1.000 y 10.000 registros. La versión concreta de Firebase (Realtime Database o Cloud Firestore), versión de Android, dispositivo y versiones de dependencias deberán quedar fijadas en el archivo de entorno antes de la medición final.

2. Marco teórico

2.1 Gestor de gastos personales y operaciones de datos

Un gestor de gastos personales es una aplicación que permite registrar transacciones como egresos, ingresos, categorías, importes, fechas y descripciones, además de recuperar información para mostrar listados o resúmenes. Para este experimento, el registro lógico mínimo contendrá identificador, tipo de movimiento, categoría, importe, fecha y descripción.

Se utilizarán datos sintéticos, sin nombres, cuentas, números de tarjeta ni información financiera real.
Las operaciones de inserción incorporan nuevos registros y las consultas recuperan uno o varios registros según una regla previamente definida. Para que la comparación sea justa, ambos almacenes deben recibir la misma cantidad y contenido lógico, y la consulta debe representar el mismo resultado esperado. Se documentarán las diferencias inevitables entre el modelo relacional de SQLite y el modelo de documentos o árbol de Firebase.

2.2 SQLite y almacenamiento local

SQLite es un motor de base de datos relacional embebido, que permite conservar datos estructurados en un archivo local y ejecutar consultas sin solicitar una respuesta a un servidor remoto. En una aplicación Android, esta arquitectura puede reducir la dependencia de la conectividad para las operaciones locales. El rendimiento concreto depende, entre otros factores, de índices, transacciones, tamaño de registros, consultas y dispositivo.
En el experimento, SQLite será el tratamiento de almacenamiento local. La inserción se considerará terminada cuando la operación de base de datos haya finalizado correctamente y la transacción, si se utiliza, se haya confirmado. La consulta se considerará terminada cuando el conjunto solicitado esté disponible para el código que realiza la medición. Se registrará la configuración relevante, incluida la política de transacciones y los índices usados.

2.3 Firebase y almacenamiento en la nube

Firebase proporciona servicios de base de datos en la nube para aplicaciones móviles. La comunicación con el servicio y el comportamiento de los SDK dependen de la configuración, la conectividad y el producto concreto seleccionado. Las funciones de caché o persistencia local pueden cambiar qué significa una operación completada, por lo que el protocolo debe especificar si mide una lectura desde caché, una respuesta del servidor o la confirmación de escritura remota.
Para este estudio, el equipo deberá declarar explícitamente si utiliza Firebase Realtime Database o Cloud Firestore. No se deben combinar resultados de ambos productos como si fueran una sola tecnología. Para inserciones, el cronómetro se detendrá con el callback de éxito definido para confirmar la operación bajo la configuración elegida. Para consultas, se especificará si se espera respuesta del servidor y se mantendrá la misma política en todas las repeticiones. Se anotará la conectividad en cada serie.

2.4 Calidad del software y eficiencia de desempeño

ISO/IEC 25010:2023 define un modelo de calidad de producto que sirve como referencia para especificar y evaluar características de productos TIC y software. En este trabajo se selecciona la eficiencia de desempeño como dimensión central, operacionalizada mediante el comportamiento temporal de inserciones y consultas. La usabilidad se considera contexto del producto, pero no se medirá formalmente en esta entrega, pues el diseño se concentra en el benchmark técnico.
El tiempo de respuesta se expresa en milisegundos. Para cada escenario se reportarán media, mediana, desviación estándar, mínimo, máximo y percentiles 90 y 95 si el tamaño de muestra lo permite. La mediana será útil ante valores extremos; la media facilitará comparaciones generales. No se afirmará que una diferencia es importante solo por observar promedios: se reportará dispersión y tamaño del efecto cuando sea viable.

2.5 Medición de rendimiento en aplicaciones

La API Performance del navegador no es el instrumento principal para medir la aplicación Android nativa. La medición se realizará en el propio código de prueba con reloj monotónico de alta resolución, por ejemplo SystemClock.elapsedRealtimeNanos() en Android, y los

resultados se exportarán a CSV. El inicio y el final de cada medición se definirán con precisión para que no se midan operaciones diferentes.
Se distinguirán las mediciones de preparación de datos de las mediciones de inserción y consulta. La carga inicial de los registros no se mezclará con las mediciones objetivo. Se limpiará o reinicializará el almacén entre escenarios cuando corresponda y se verificará la cantidad final de registros. Las pruebas que fallen o devuelvan resultados incorrectos se registrarán, no se eliminarán silenciosamente.

2.6 Seguridad y ética

Los registros de gastos pueden revelar hábitos personales. Por ello, se emplearán datos sintéticos y credenciales de prueba. Las claves de Firebase no deben confundirse con secretos de servidor, pero las reglas de seguridad deben impedir acceso no autorizado a datos de prueba; no se publicarán credenciales privadas ni archivos de configuración con secretos. OWASP Top 10 ofrece una referencia de riesgos comunes de aplicaciones web y servicios conectados, incluyendo control de acceso, fallos criptográficos e inyección; su uso aquí es orientativo y no sustituye una auditoría de seguridad.
El equipo declarará el uso de asistentes de inteligencia artificial de acuerdo con las instrucciones de la asignatura. Toda referencia bibliográfica debe verificarse y toda decisión de diseño debe ser comprendida por los integrantes. No se recopilarán datos de personas ni se harán afirmaciones sobre mejora de hábitos de ahorro, ya que el experimento mide desempeño técnico, no efectos conductuales.

3. Estado del arte

3.1 Estrategia de búsqueda

La revisión se plantea como una búsqueda documentada de literatura técnica y académica, siguiendo la recomendación general de definir preguntas, criterios de selección, extracción y síntesis antes de redactar conclusiones. Se consultaron fuentes web académicas y documentación oficial el 11 de octubre de 2026. Las bases objetivo para completar la búsqueda son Google Académico, DOAJ, ScienceDirect, IEEE Xplore y el portal editorial de las revistas localizadas.
Cadena principal: (SQLite OR Firebase OR "Cloud Firestore" OR "Realtime Database") AND (Android OR mobile) AND (performance OR latency OR "response time") AND (insert OR query OR CRUD). Cadena complementaria: ("mobile database" OR "cloud database") AND (benchmark OR comparison) AND (SQLite OR Firebase). Para calidad: ("ISO/IEC 25010" AND "performance efficiency" AND software).
Criterios de inclusión: (a) publicación académica o documentación oficial pertinente; (b) describe arquitectura, método, métrica o resultado de rendimiento de bases móviles; (c) permite identificar autores, año y fuente; (d) prioridad a publicaciones 2021–2026. Criterios de exclusión: (a) duplicados; (b) textos sin autoría o procedencia verificable; (c) publicidad sin método; (d) fuentes que no traten bases móviles, rendimiento o calidad de software. El número de resultados por base debe completarse en el registro de búsqueda del repositorio después de ejecutar la cadena en cada plataforma; no se inventan conteos de resultados.

3.2 Matriz comparativa de antecedentes

Antecedente 1 — Kabakuş (2019): “A Performance Comparison of SQLite and Firebase Databases from a Practical Perspective”. El estudio implementó experimentos con operaciones comunes y distintos tamaños de datos; informa diferencias dependientes de operación y volumen y reporta ventaja de SQLite en varias operaciones, con una excepción para eliminación. Aporta un antecedente directamente relacionado con la comparación, pero es anterior al periodo preferido de cinco años y las configuraciones actuales pueden

haber cambiado. Vacío: el presente proyecto restringe el alcance a inserción y consulta y documenta un protocolo replicable en el contexto específico de un gestor de gastos.
Antecedente 2 — Kot y Smołka (2023): “A performance analysis of a cloud database on mobile devices”. Compara Firebase Realtime Database y Cloud Firestore con SQLite, considerando operaciones CRUD, número y tamaño de registros y complejidad de estructura. Los autores informan que la estabilidad de la red y la distancia al servidor influyen en la precisión y que las bases locales y cloud pueden comportarse de manera diferente según la operación y el volumen. Vacío: hace falta un protocolo alineado con los datos y las operaciones del gestor de gastos del equipo, con semilla de datos, repeticiones y archivos de salida abiertos.
Antecedente 3 — Erike et al. (2025, verificar metadatos finales): “A Comparative Performance Evaluation of SQLite, MySQL, and Firebase for Modern Application Development Using a Parallel Execution Approach”. El resumen disponible reporta comparación de tiempos en operaciones y tipos de carga distintos. Aporta evidencia reciente de que el orden de rendimiento puede variar por operación y contenido. Vacío: el entorno y enfoque descritos no sustituyen un experimento Android controlado con volúmenes definidos para gastos; antes de citarlo en la entrega final, verificar volumen, páginas y DOI en la publicación original.
Antecedente 4 — Ara, Sik-Lanyi y Kelemen (2024): “Accessibility engineering in web evaluation process: a systematic literature review”. Aunque se enfoca en accesibilidad y no en bases de datos, documenta que la evaluación de aplicaciones debe planificarse con métodos explícitos y que la literatura aplica distintas técnicas. Su aporte es metodológico y contextual, no evidencia de superioridad de SQLite o Firebase. Vacío: el presente estudio no evaluará accesibilidad; delimita una métrica técnica reproducible.
Antecedente 5 — Kitchenham y Charters (2007): “Guidelines for performing Systematic Literature Reviews in Software Engineering”. Propone planificar, conducir y reportar revisiones con protocolo explícito. Es una referencia metodológica clásica, aunque fuera de la ventana de actualidad exigida. Aporta la estructura para documentar cadenas de búsqueda, criterios y síntesis. Vacío: no compara tecnologías de almacenamiento; se usa para el método de revisión, no como evidencia empírica de rendimiento.
Antecedente 6 — ISO/IEC 25010:2023. Define un modelo de calidad de producto con características y subcaracterísticas que sirven como referencia para especificar y evaluar calidad. Aporta un vocabulario formal para situar la eficiencia de desempeño. Vacío: el estándar no aporta resultados experimentales específicos sobre SQLite o Firebase; el equipo debe operacionalizar la característica en métricas propias.
Síntesis crítica: los antecedentes directamente comparativos muestran que la latencia depende de la operación, el tamaño de los datos y el entorno, y que una base local y un servicio cloud no deben interpretarse como arquitecturas equivalentes. La brecha del proyecto no es afirmar que no existen comparaciones, sino generar evidencia reproducible para el prototipo y las condiciones que el equipo puede controlar: mismo dispositivo Android, estructura de datos equivalente, tres volúmenes, operaciones de inserción y consulta, treinta repeticiones y registro de conectividad. Los resultados deberán interpretarse como evidencia local del experimento, no como una clasificación universal.

4. Diseño metodológico

4.1 Enfoque y tipo de investigación

Enfoque cuantitativo, de tipo aplicado y diseño experimental comparativo. Se manipularán dos factores principales: tecnología de almacenamiento y volumen de datos; la operación se tratará como factor adicional o se analizará por separado. La variable de respuesta principal

será el tiempo de respuesta en milisegundos. El experimento no medirá directamente ahorro personal ni satisfacción de usuarios.

4.2 Variables y operacionalización

Variable independiente 1: tecnología de almacenamiento, con niveles SQLite local y Firebase (producto específico declarado antes de iniciar). Variable independiente 2: volumen de registros, con niveles 100, 1.000 y 10.000. Variable independiente 3: operación, con niveles inserción y consulta.

Variable dependiente: tiempo de respuesta por operación, en milisegundos. Definición operacional: diferencia entre el instante monotónico registrado al iniciar la operación y el instante en que se cumple el criterio de finalización documentado. Instrumento: cronómetro monotónico integrado al prototipo Android y archivo CSV de resultados.

Variables controladas: dispositivo físico o emulador (se elegirá uno y no se alternará durante una serie), versión de Android, versión de la app, versiones de dependencias, esquema lógico de registros, orden de campos, política de índices, tamaño aproximado de cada registro, estrategia de confirmación, condiciones de red y estado inicial del almacenamiento. Variables registradas como covariables: latencia/estado de red en Firebase, errores, número de registros resultantes y hora de ejecución.

4.3 Protocolo experimental reproducible

1. Registrar en docs/entorno.md el modelo del dispositivo, CPU, RAM, versión de Android, versión de Android Studio/Gradle, versiones de SDK y librerías, producto Firebase utilizado y configuración de red. 2. Generar un conjunto determinista de gastos sintéticos con semilla fija 42. 3. Validar que los registros lógicos equivalentes se cargan en ambos almacenes. 4. Preparar los escenarios con 100, 1.000 y 10.000 registros; la carga de preparación no se incluye en los tiempos objetivo. 5. Ejecutar cinco repeticiones de calentamiento por escenario y operación. 6. Ejecutar treinta mediciones por combinación de tecnología, volumen y operación. 7. Aleatorizar el orden de los escenarios con semilla 42 para reducir sesgo por calentamiento, batería o cambios temporales. 8. Guardar una fila por medición en CSV. 9. Verificar conteos y contenido de la consulta; registrar errores y resultados inválidos. 10. Repetir una serie si hubo una interrupción externa documentada, sin borrar el registro original. 11. Calcular estadísticas descriptivas y pruebas de comparación sobre las observaciones válidas.

Para la inserción, se usará el mismo conjunto de registros y el cronómetro terminará al completarse la operación según el criterio definido para cada almacén. Para la consulta, se usará una consulta equivalente, por ejemplo recuperar los registros del conjunto de prueba o recuperar una fracción definida con un filtro por fecha/categoría. La consulta exacta se fijará antes de la medición. No se comparará una consulta de todos los registros en SQLite con una consulta limitada en Firebase.

Para evitar que la caché cambie inadvertidamente el significado de la prueba, el equipo debe decidir y documentar si las consultas Firebase exigen confirmación del servidor o admiten caché local. No se mezclarán ambas modalidades dentro de la misma serie. Si el prototipo no permite fijar esta condición, se anotará como limitación y se reportarán por separado las mediciones identificadas como caché y red.

4.4 Población, muestra y corpus

No se reclutarán personas como participantes. La unidad de análisis será cada ejecución de inserción o consulta. La muestra experimental estará formada por 2 tecnologías × 3 volúmenes × 2 operaciones × 30 mediciones = 360 observaciones principales, más las ejecuciones de calentamiento, siempre que todas las combinaciones puedan ejecutarse. Las ejecuciones de calentamiento se conservarán en un archivo separado y no se incluirán en los

cálculos principales. Si un escenario falla, se documentará y no se sustituirá por una medición inventada.

4.5 Plan de análisis

Para cada combinación se calcularán media, mediana, desviación estándar, mínimo, máximo y percentiles 90 y 95. Se utilizarán diagramas de caja para detectar dispersión y posibles valores atípicos. Los valores atípicos no se eliminarán automáticamente; cualquier exclusión deberá tener una regla previa y una justificación verificable, y se reportarán análisis con y sin exclusiones si fuera necesario.

Como los tiempos pueden no seguir una distribución normal, se examinarán histogramas y gráficos Q-Q, además de una prueba de normalidad si el tamaño y las condiciones lo permiten. Para dos tecnologías en escenarios comparables, se considerará una prueba t si se cumplen sus supuestos o Mann–Whitney si no se cumplen; cuando se hagan comparaciones pareadas por bloques de escenario, se valorará una prueba pareada apropiada. Si se evalúan varios volúmenes simultáneamente, se considerará ANOVA o Kruskal–Wallis y ajuste por comparaciones múltiples. Se fijará $\alpha = 0{,}05$ y se reportará tamaño del efecto e intervalos de confianza cuando sea viable. La elección final de prueba dependerá de la distribución y estructura real de los datos.

Se reportarán las diferencias en milisegundos y el cambio porcentual respecto a la otra tecnología. No se afirmará causalidad fuera de los factores manipulados ni se extrapolarán resultados a otros dispositivos, redes, versiones de Firebase o tamaños de datos.

4.6 Amenazas a la validez y mitigación

Validez interna: otras tareas del dispositivo, calentamiento, ahorro de batería y variación de red pueden afectar los tiempos. Mitigación: usar el mismo dispositivo, cerrar aplicaciones ajenas, mantener condiciones de batería y temperatura tan estables como sea posible, aleatorizar escenarios y registrar interrupciones.

Validez de constructo: el tiempo medido podría no representar la experiencia completa del usuario. Mitigación: definir inicio y fin de la medición y no mezclar tiempo de interfaz con tiempo de base de datos; reconocer que no se evalúa usabilidad formal.

Validez externa: los resultados pueden depender del modelo de dispositivo, proveedor de red, región del servidor, estructura de datos y versión de Firebase. Mitigación: describir el entorno y limitar las conclusiones al escenario probado.

Validez de conclusión: pocas repeticiones, datos atípicos o pruebas estadísticas inadecuadas pueden conducir a conclusiones débiles. Mitigación: treinta mediciones por combinación, análisis de dispersión, elección de pruebas según supuestos y reporte de tamaños de efecto.

Validez de reproducibilidad: cambios no documentados en dependencias o reglas de Firebase dificultan replicar el estudio. Mitigación: fijar versiones, registrar configuración y publicar scripts de generación de datos y análisis sin credenciales privadas.

4.7 Consideraciones éticas

Se usarán datos sintéticos y no se recopilarán datos personales ni financieros reales. Las reglas de acceso de Firebase se configurarán para un entorno de prueba y no se dejarán bases abiertas públicamente. Las credenciales privadas y archivos locales de configuración se excluirán del repositorio. El equipo reconocerá las licencias de las herramientas y fuentes consultadas, citará los antecedentes y declarará el uso de inteligencia artificial para apoyar la organización, redacción o revisión del documento, verificando manualmente las afirmaciones y referencias.

5. Prototipo y evidencia técnica en GitHub

El repositorio público consultado contiene un proyecto Android basado en Gradle/Kotlin y un README que define el objetivo de comparar Firebase y SQLite. Por ello, el prototipo debe continuar en ese entorno; no debe reemplazarse por una aplicación PHP/MySQL sin una decisión explícita del equipo. La página del repositorio muestra carpetas app, Docs y gradle/wrapper, y los archivos build.gradle.kts, settings.gradle.kts, README.md y .gitignore. La inspección web no confirma por sí sola que el proyecto compile o que la integración continua esté en verde.

Elementos que deben agregarse o verificarse: (1) docs/estado-del-arte.md con cadenas, bases, fechas, conteos reales y matriz de antecedentes; (2) docs/entorno.md con versiones exactas; (3) docs/adr/0001-eleccion-de-bd.md; (4) workflow .github/workflows/ci.yml para compilar y ejecutar pruebas; (5) una prueba automatizada mínima; (6) issues por objetivo específico; (7) etiqueta v0.2-marco-metodologia; (8) evidencia de Actions en verde.

El archivo de integración continua debe adaptarse a la versión de Gradle y Java/Kotlin que realmente utiliza el repositorio. No se debe copiar a ciegas un workflow de Maven, porque el repositorio observado es un proyecto Android con Gradle. Antes de crear el workflow definitivo, confirmar el nombre de la tarea Gradle que compila y prueba el módulo app.

6. Registro de decisión arquitectónica (ADR 0001)

Estado: propuesta pendiente de aprobación del equipo. Contexto: el prototipo requiere guardar y consultar registros de gastos y comparar rendimiento de almacenamiento local y cloud. Opciones consideradas: (A) SQLite local; (B) Firebase Realtime Database; (C) Cloud Firestore; (D) combinación de almacenamiento local y sincronización cloud. Decisión para el experimento: comparar SQLite con un solo producto Firebase, elegido y declarado antes de las mediciones, para evitar mezclar resultados de productos diferentes. La elección de Firebase Realtime Database o Cloud Firestore debe confirmarse revisando el código y la configuración actual del repositorio. Consecuencias: se pueden comparar tiempos bajo condiciones definidas; el resultado no representará por sí solo coste, seguridad, escalabilidad ni experiencia total. Se deberán proteger las credenciales y documentar el impacto de red y caché.

7. Conclusiones metodológicas

La evidencia disponible justifica realizar una comparación controlada, pero no permite anticipar qué tecnología ganará en el prototipo del equipo. Los estudios previos indican que los resultados dependen de operación, volumen, red y arquitectura. El aporte de esta investigación será un protocolo verificable, con datos sintéticos, condiciones declaradas, múltiples repeticiones y resultados reproducibles. La conclusión sobre rendimiento se redactará después de obtener las mediciones; no se deben presentar resultados simulados como resultados experimentales.

Referencias

Kabakuş, A. T. (2019). A Performance Comparison of SQLite and Firebase Databases from a Practical Perspective. Düzce Üniversitesi Bilim ve Teknoloji Dergisi, 7(1), 314–325. https://doi.org/10.29130/dubited.441672

Kot, S., & Smołka, J. (2023). A performance analysis of a cloud database on mobile devices. Journal of Computer Sciences Institute, 29, 360–365. https://doi.org/10.35784/jcsi.3798

Erike, A. I., Alameziem, M.-C. C., Elei, F. O., Ovwonuri, A. O., & Nwandu, I. C. (2025). A Comparative Performance Evaluation of SQLite, MySQL, and Firebase for Modern

Application Development Using a Parallel Execution Approach. UNIZIK Journal of Engineering and Applied Sciences. (Verificar volumen, páginas y DOI en la versión editorial antes de la entrega final).

Ara, J., Sik-Lanyi, C., & Kelemen, A. (2024). Accessibility engineering in web evaluation process: a systematic literature review. Universal Access in the Information Society, 23, 653–686. https://doi.org/10.1007/s10209-023-00967-2

ISO/IEC. (2023). ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. https://www.iso.org/standard/78176.html

OWASP Foundation. (2025). OWASP Top 10. https://owasp.org/www-project-top-ten/ (Consultar la versión vigente; la edición 2025 reemplaza la edición 2021).

Google. (s. f.). Firebase Realtime Database documentation. https://firebase.google.com/docs/database

Google. (s. f.). Cloud Firestore documentation. https://firebase.google.com/docs/firestore

Android Developers. (s. f.). Save data in a local database using Room. https://developer.android.com/training/data-storage/room

SQLite Consortium. (s. f.). About SQLite. https://www.sqlite.org/about.html

Kitchenham, B., & Charters, S. (2007). Guidelines for performing Systematic Literature Reviews in Software Engineering (EBSE Technical Report EBSE-2007-01). Keele University and Durham University. https://ebse.webspace.durham.ac.uk/ebse-bibliography/guidelines-for-performing-systematic-literature-reviews-in-software-engineering/

GitHub. (s. f.). Building and testing Java with Gradle. GitHub Actions documentation. https://docs.github.com/actions/automating-builds-and-tests/building-and-testing-java-with-gradle

Android Developers. (s. f.). Measure app performance. https://developer.android.com/topic/performance